When a `ReactiveAuditorAware` bean is defined and auditing is enabled, Spring Data will use it to extract the username of the currently authenticated user represented as a `String`. Even in this case, we can use `@CreatedBy` and `@LastModifiedBy` to annotate two new fields in the `Order` record. They will be populated by Spring Data automatically whenever a create or update operation is performed on the entity.

Listing 12.27 Fields to capture user audit metadata in an R2DBC entity

```
@Table("orders")
public record Order (
  ...

  @CreatedBy                    ❶
  String createdBy,

  @LastModifiedBy               ❷
  String lastModifiedBy,

){
  public static Order of(String bookIsbn, String bookName,
    Double bookPrice, Integer quantity, OrderStatus status
  ) {
    return new Order(null, bookIsbn, bookName, bookPrice, quantity, status,
      null, null, null, null, 0);
  }
}
```

❶ Who created the entity

❷ Who modified the entity the last time

After adding the new fields, we need to update a few classes using the `Order` all-args constructor, which now requires you to pass values for `createdBy` and `lastModifiedBy`.

The `OrderService` class contains the logic for updating dispatched orders. Open it and change the `buildDispatchedOrder()` method to ensure the audit metadata is correctly passed along when calling the data layer.